



Explorer's Workbook: Kubernetes RBAC (Fresher Edition)

Idhu lecture illa — idhu oru expedition. Every experiment: **predict pannu** → **run pannu** → **observe pannu** → **explain pannu**. Wrong prediction OK — adhu dhaan learning.
AWS IAM theriஞ்சுவங்களukku: ovvoru section-la oru  **IAM Corner** irukku — RBAC concept-ah IAM-oda compare pannu explain pannum.

Your name: _____ Date: _____

Idhu yaaru-ku?

- Basic Kubernetes theriஞ்சுவங்க (pods, namespaces enna nu theriyum).
 - RBAC-ku fresher.
 - minikube cluster kaiyila irukkuravங்க.
-

How to Use This Workbook

Box	Enna pannanum
 PREDICT	Run panradhukku munnadi, enna aagum nu yosichi ezhuthu.
 RUN	Commands-ah exact-ah run pannu.
 OBSERVE	Real-ah enna aachu nu ezhuthu.
 EXPLAIN	Yen apdi aachu? Prediction match aachaa?

 **Dig Deeper** = early-ah mudichaa, extra challenge.

Part 0 — RBAC enna? (idhai padichuko)

RBAC = Role-Based Access Control. Kubernetes-oda security system. Oru simple question-ku answer dhaan RBAC: "Yaaru, enna pannalaam?"

Cluster-la nariya per work pannuvanga — developers, testers, admins, apps.
Ellarukum full access kuduthaa, oru chinna mistake periya problem aagidum.
RBAC limit pannidum — overuthar enna pannanumo adhu mattum.

RBAC-oda 4 main pieces

Piece	Enna pannum	Scope
Role	"Enna pannalaam" — permission list	Oru namespace mattum
ClusterRole	Same, aana cluster-wide	Full cluster
RoleBinding	Role-ah oru user-ku connect pannum	Oru namespace mattum
ClusterRoleBinding	ClusterRole-ah cluster-wide connect pannum	Full cluster

Golden rule: **Role** = "enna pannalaam". **Binding** = "yaaru pannalaam". **Rendum seranthaal dhaan** oru user-ku access varum.

 IAM Corner

AWS IAM theri  RBAC easy:

AWS IAM	Kubernetes RBAC
IAM Policy	Role / ClusterRole
IAM User / Role	User / ServiceAccount
Policy-ah user-ku attach panradhu	RoleBinding / ClusterRoleBinding
Action (s3:GetObject)	verbs (get, list)
Resource (bucket ARN)	resources (pods, secrets)

Periya difference: IAM-la `"Effect": "Deny"` irukku. RBAC-la **Deny illave illa** — by default ellaam blocked, neenga `Allow` rules mattum add panreenga. Allowlist model.

Setup — Lab Ready Pannu

minikube start aagirukaa nu check pannu:

```
bash
minikube status
kubectl get nodes
```

Lab-ku oru namespace create pannu:

```
bash  
  
kubectl create namespace rbac-lab
```

 Idha namespace-la dhaan namma Role/RoleBinding test panna porom.
Namespace-na cluster-kulla oru "section" — IAM-la idhu maathiri concept illa.

Experiment 1 — "Naan enna pannalaam?" — `kubectl auth can-i`

RBAC test panna oru semma command irukku — `kubectl auth can-i`.

PREDICT

`kubectl auth can-i` — neenga oru action pannamudiyumaa nu sollum. Neenga ippo cluster admin (minikube default-ah admin access tharum). `kubectl auth can-i delete pods` run panna, enna varum nu nenkareenga? (`yes`)/(`no`) Write: _____

RUN

```
bash  
  
kubectl auth can-i delete pods  
kubectl auth can-i create deployments  
kubectl auth can-i get secrets --namespace rbac-lab
```

OBSERVE

 Moonu command-um enna sonnuchu? _____

EXPLAIN

 Ellaam (`yes`) vandhuchaa? Yen-na minikube unga default user-ku full admin access tharudhu. Idhu next experiments-la important — kashta-padaama test panna idha use pannuvom.

 **IAM Corner:** `kubectl auth can-i` = AWS IAM Policy Simulator maathiri. "Indha identity indha action panna mudiyumaa" nu check panra tool.

Experiment 2 — Oru Role Create Pannu (= IAM read-only policy)

Oru **Role** = permission list. Namma "pods-ah read panna mattum" Role pannalaam.

PREDICT

Role create panna mattum, andha permission yaarukkavadhu kedaikumaa?

Illa, vera onnu kooda pannanumaa? Write your guess: _____

RUN

`role-pod-reader.yaml` create pannu:

yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: rbac-lab
  name: pod-reader
rules:
- apiGroups: [""]           # "" = core API group (pods idhula varum)
  resources: ["pods"]       # edhu mela - ~ IAM "Resource"
  verbs: ["get", "list", "watch"] # enna pannalaam - ~ IAM "Action"
```

bash

```
kubectl apply -f role-pod-reader.yaml
kubectl get role -n rbac-lab
kubectl describe role pod-reader -n rbac-lab
```

OBSERVE

 `describe` output-la, `pod-reader` enna verbs allow pannudhu? _____  `delete` andha list-la irukaa? _____

EXPLAIN

 Role create aachu — aana idhu *innum* yaarukum apply aagala. Yen? (Hint: golden rule — Role + _____ rendum venum.)

 **IAM Corner:** Indha Role = IAM-la oru policy ezhuthi vechathu maathiri. Policy ezhuthi vechaa mattum velai aagaadhu — yaarukavadhu **attach** pannanum. Adhu dhaan next experiment.

Field notes:

- `apiGroups: [""]` → core resources (pods, services). Deployments maathiri-na `apiGroups: ["apps"]`.
 - `verbs` → `get`, `list`, `watch`, `create`, `update`, `delete` — idhu actions.
 - `resources` → `pods`, `secrets`, `deployments` — idhu objects.
-

Experiment 3 — RoleBinding: Role-ah User-ku Connect Pannu

Ippo "yaaru" part. Namma oru test user — `intern` — ku Role-ah bind pannalaam.

PREDICT

RoleBinding `pod-reader` Role-ah `intern`-ku connect pannum. Apparam `intern` pods *list* panna mudiyum nu nenikkireenga? `intern` pods *delete* panna mudiyumaa? Write your guesses: _____

RUN

`rolebinding-intern.yaml` create pannu:

yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: intern-pod-reader
  namespace: rbac-lab
subjects:
- kind: User
  name: intern          # yaaru-ku — andha user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader      # endha Role — Experiment 2-la create pannadhu
  apiGroup: rbac.authorization.k8s.io
```

bash

```
kubectl apply -f rolebinding-intern.yaml
kubectl get rolebinding -n rbac-lab
```

Ippo `intern`-ah maathiri test pannu (`--as` flag user-ah impersonate pannum):

```
bash
```

```
kubectl auth can-i list pods --namespace rbac-lab --as intern
kubectl auth can-i delete pods --namespace rbac-lab --as intern
kubectl auth can-i get secrets --namespace rbac-lab --as intern
```

👁️ OBSERVE

 `list pods` → _____ `delete pods` → _____ `get secrets` → _____

💡 EXPLAIN

 `list pods` `yes` vandhirukum, aana `delete pods` `no`. Yen `delete` `no` vandhuchu? (Hint: Experiment 2-la Role-la `delete` verb add pannala.)

 `get secrets` yen `no`? (Hint: Role-la `resources` ku `pods` mattum dhaan, `secrets` illa.)

 **Big idea:** RBAC-la default-ah ellaam `no`. Neenga explicit-ah `Allow` panna mattum dhaan `yes`. Idhu enna model nu Part 0-la padichom?

 **IAM Corner:** `--as intern` = IAM-la "assume role" / impersonation maathiri — innoru identity-ah maathiri test panradhu. RoleBinding = policy-ah user-ku attach panra step.

🔧 Experiment 4 — Role Edit Panni Access Maathu

RBAC live-ah work aagum. Role-ah maathina, access udane maarum.

😬 PREDICT

`pod-reader` Role-la `delete` verb add panna porom. Add panna udane, `intern` pods delete panna mudiyumaa — pod restart venumaa, illa udane velai aagidumaa? Write your guess:

▶️ RUN

```
bash

# delete verb add pannu
kubectl edit role pod-reader -n rbac-lab
```

Editor-la `verbs` line-ah idhu maathiri maathu, save panni exit pannu:

```
yaml
```

```
verbs: ["get", "list", "watch", "delete"]
```

```
bash
```

```
kubectl auth can-i delete pods --namespace rbac-lab --as intern
```

🔊 OBSERVE

✏️ Ippo `delete pods` enna sonnuchu? _____

💡 EXPLAIN

✏️ Role edit pannina udane access maaruchu — nothing restart pannala.

Idhu yen useful nu oru sentence-la sollu:

🔍 Dig Deeper

`pod-reader`-la `secrets` resource add pannu. Apparam `intern` secrets read panna mudiyumaa nu test pannu. Verb add panna `get`, resource add panna `get` — rendum puriyutha?

🧪 Experiment 5 — ClusterRole: Namespace Border Thaandi

`Role` oru namespace-ku mattum. `ClusterRole` full cluster-ku. Sila resources (nodes maathiri) edha namespace-laiyum illa — adhukku ClusterRole kandippa venum.

😬 PREDICT

`intern`-ku ippo `rbac-lab` namespace-la dhaan pod access irukku. `intern` **nodes** (cluster-level resource) list panna mudiyumaa nu nenikkireenga? Write your guess: _____

▶️ RUN

First confirm intern nodes paaka mudiyaadhu:

```
bash
```

```
kubectl auth can-i list nodes --as intern
```

`clusterrole-node-reader.yaml` create pannu:

```
yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list"]
```

```
clusterrolebinding-intern.yaml
```

 create pannu:

```
yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: intern-node-reader
subjects:
  - kind: User
    name: intern
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

```
bash

kubectl apply -f clusterrole-node-reader.yaml
kubectl apply -f clusterrolebinding-intern.yaml


kubectl auth can-i list nodes --as intern
```

 OBSERVE

ClusterRoleBinding panradhukku munnadi `list nodes` → _____ Apram → _____

 EXPLAIN

 **Role** use pannama **ClusterRole** yen vᱫᱽndi vandhuchu? (Hint: nodes edha namespace-kulla irukaa?) _____

IAM Corner: Role+RoleBinding = oru AWS-region/section-ku mattum apply aagura policy maathiri. ClusterRole+ClusterRoleBinding = full AWS account-ku apply aagura policy

maathiri — IAM default behaviour idhu.

Compare panni nyabagam vechuko:

	Role + RoleBinding	ClusterRole + ClusterRoleBinding
Scope	Oru namespace	Full cluster
Resources	Namespaced (pods, secrets)	Cluster-level kooda (nodes)
Example	"rbac-lab-ls pods read"	"ellaa nodes-um read"

Experiment 6 — ServiceAccount: Apps-ku Identity (= IAM Role for EC2)

Ippo varai *human user* (`intern`). Aana **pods kooda** API call pannum. Pod-ku identity venum — adhu **ServiceAccount**.

PREDICT

Oru pod default-ah (`default`) ServiceAccount use pannum, adhukku konjam kooda permission illa. Naama oru ServiceAccount create panni, athukku (`pod-reader`) Role bind pannina, andha SA use panra pod pods list panna mudiyumaa? Write your guess: _____

RUN

ServiceAccount create pannu:

```
bash
kubectl create serviceaccount app-sa -n rbac-lab
```

`rolebinding-sa.yaml` create pannu (Role same `pod-reader`, subject SA):

```
yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-sa-pod-reader
  namespace: rbac-lab
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: rbac-lab
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```
bash

kubectl apply -f rolebinding-sa.yaml

# ServiceAccount-ah maathiri test pannu
kubectl auth can-i list pods -n rbac-lab \
  --as=system:serviceaccount:rbac-lab:app-sa
kubectl auth can-i delete pods -n rbac-lab \
  --as=system:serviceaccount:rbac-lab:app-sa
```

👁 OBSERVE

✏ SA-ku `list pods` → _____ `delete pods` → _____

💡 EXPLAIN

✏ `User` subject-ku bind panna mudiyum, `ServiceAccount` subject-ku-um bind panna mudiyum. Rendukum வித்துயாசம் enna? (Hint: oruthar human, innoru...?)

🟠 **IAM Corner:** ServiceAccount = AWS-la EC2 instance illa Lambda-ku attach panra **IAM Role** maathiri. Human illa — application-ku identity.

`system:serviceaccount:NAMESPACE:NAME` format dhaan SA-oda full "username".

🔍 Dig Deeper

Oru real pod-la `app-sa`-ah use pannu (`spec.serviceAccountName: app-sa`), pod-kulla irundhu `kubectl get pods` try pannu. (Pod-la kubectl venum, illa API-ah curl pannu.)

Ellaam neengaye scratch-la ezhutha v_ondaam. Kubernetes ready-made ClusterRoles tharum.

 PREDICT

`view`, `edit`, `admin`, `cluster-admin` — idhu built-in ClusterRoles. `view` ClusterRole enna allow pannum nu nenikkireenga — read mattum-aa, illa write-um-aa? Write your guess:

 **RUN**

```
bash

# built-in ClusterRoles paaru
kubectl get clusterrole | grep -E '^(view|edit|admin|cluster-admin)'

# view ClusterRole-la enna verbs irukku nu paaru
kubectl describe clusterrole view | head -n 30
```

👁️ OBSERVE

 `view` ClusterRole-la `create`/`delete` verbs irukaa, illa `get`/`list` mattum-aa?

EXPLAIN

 Scratch-la ezhuthaama indha built-in roles use pannina enna advantage?

■ **IAM Corner:** Idhu AWS managed policies maathiri — `ReadOnlyAccess`, `AdministratorAccess`. AWS ready-ah tharum, neenga attach mattum pannanum. Kubernetes-la: `view` $\approx$ `ReadOnlyAccess`, `cluster-admin` $\approx$ `AdministratorAccess`.

Built-in ClusterRoles quick guide:

ClusterRole	Enna tharum	IAM equivalent
<code>view</code>	Read-only (secrets thavira)	<code>ReadOnlyAccess</code>
<code>edit</code>	Read + write (RBAC thavira)	PowerUser maathiri
<code>admin</code>	Namespace-la full (RBAC kooda)	namespace admin
<code>cluster-admin</code>	Cluster-la ellaam	<code>AdministratorAccess</code>

⚠️ `cluster-admin` carefully use pannu — idhu "god mode". Production-la romba kammi per-ku mattum.

❧ Final Challenge — Show What You Know

Earlier pages paakaama, சொந்த words-la answer pannu:

1. RBAC-la "Role" enna pannum, "RoleBinding" enna pannum? வித்தியாசம் sollu.

 2. Oru user-ku `list pods` `yes` aana `delete pods` `no` — Role-la enna irundhuchu, enna illama irundhuchu?

 3. `Role` vs `ClusterRole` — eppo edha use pannanum?

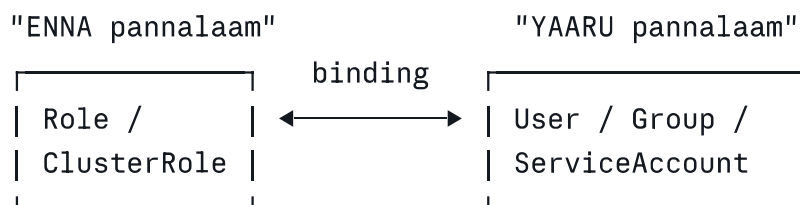
 4. RBAC-la `Deny` rule illa. Adha vechi — oru user default-ah enna access vechirupaanga?

 5. **ServiceAccount** edhuku — `User` ku enna difference?

 6. AWS IAM mapping fill pannu:
 - IAM Policy ≈ _____
 - IAM policy attachment ≈ _____
 - IAM `Action` ≈ _____
 - IAM Role for EC2 ≈ _____
-

🎨 Concept Recap

RBAC velai panra vidham:



Rendum sernthaal dhaan = ACCESS

4 pieces:

Permission ("enna")	Binding ("yaaru")	Scope
Role	RoleBinding	Oru namespace
ClusterRole	ClusterRoleBinding	Full cluster

Tip: ClusterRole-ah oru *RoleBinding*-la kooda use pannalaam — appo andha cluster-wide permission oru namespace-ku mattum apply aagum. (Idhu advanced — innum oru naal explore pannu.)

Test panra commands:

```
bash

kubectl auth can-i VERB RESOURCE                # neenga
kubectl auth can-i VERB RESOURCE --as USER      # innoru user maathiri
kubectl auth can-i VERB RESOURCE \
  --as=system:serviceaccount:NS:NAME             # serviceaccount maathiri
```

Maranthudaadha 3 truths:

1. Role = "enna", Binding = "yaaru". Rendum venum, illana access illa.
2. RBAC-la **Deny** illa — default-ah ellaam blocked, **Allow** mattum add panreenga.
3. Role = namespace, ClusterRole = full cluster.

Pack Up — Lab Clean Pannu

```
bash

kubectl delete namespace rbac-lab
kubectl delete clusterrole node-reader
kubectl delete clusterrolebinding intern-node-reader
rm -f role-pod-reader.yaml rolebinding-intern.yaml \
  clusterrole-node-reader.yaml clusterrolebinding-intern.yaml \
  rolebinding-sa.yaml
```

 Role/RoleBinding namespace-kulla — namespace delete panna pochu. Aana ClusterRole/ClusterRoleBinding cluster-wide — separate-ah delete pannanum. Idhu Experiment 5-oda lesson-ah confirm pannudhu!

Explorer's Quick Reference

```
bash
```

```
# Paaru
```

```
kubectl get role,rolebinding -n NAMESPACE
```

```
kubectl get clusterrole,clusterrolebinding
```

```
kubectl describe role NAME -n NAMESPACE
```

```
# Test
```

```
kubectl auth can-i VERB RESOURCE --as USER -n NAMESPACE
```

```
# Create (imperative – fast-ah test panna)
```

```
kubectl create role NAME --verb=get,list --resource=pods -n NS
```

```
kubectl create rolebinding NAME --role=ROLE --user=USER -n NS
```

Golden rule: Permission (Role) + Identity (Binding) = Access. Edhuvum onnu kammi-na → access illa.

Semma, Explorer! Neenga RBAC-ah padichu mattum theriஞ்சுகலா — சொந்த kaiyaala minikube-la experiment panni proof pannitteenga. 🎉